



6장. 합성곱 신경망 II

2팀 안예은, 진웨이안, 신예나

목차

#01 ResNET (1)

#02 ResNET(2)

#03 객체 인식을 위한 신경망 / 이미지 분할을 위한 신경망



01 ResNet (1)



#01 ResNet의 필요성 및 탄생 배경

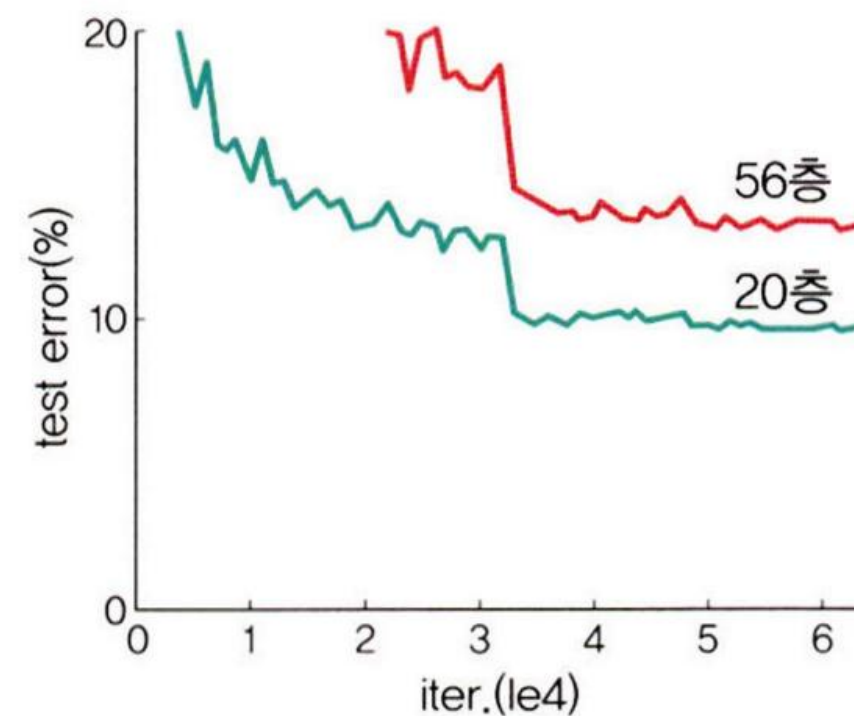
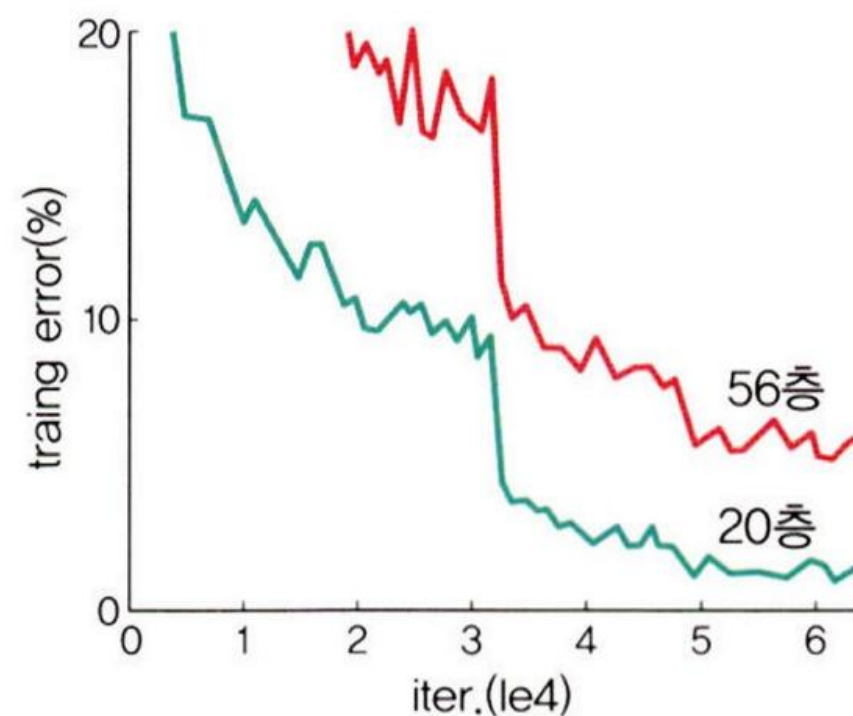
- 탄생 배경

: 마이크로소프트에서 개발한 알고리즘

“Deep Residual Learning for Image Recognition” 논문으로 발표

⇒ 신경망은 깊이가 깊어질수록 성능이 좋아지다 일정 단계 도달 시 나빠짐

▼ 그림 6-24 네트워크 56층이 20층보다 더 나쁜 성능을 보임



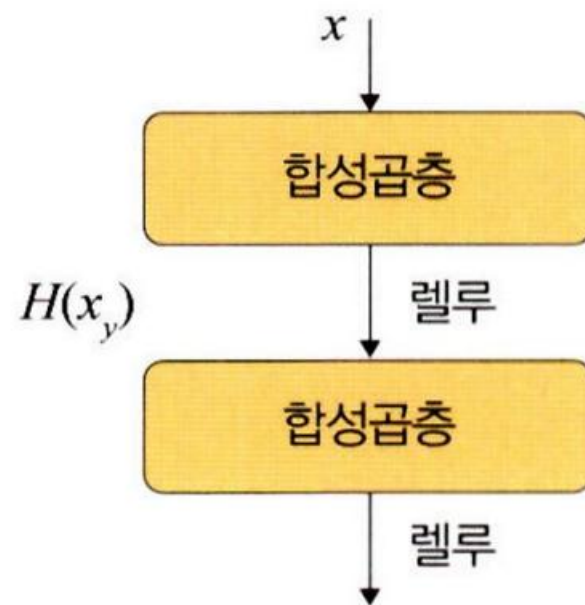
#01 ResNet의 필요성 및 탄생 배경

- 필요성

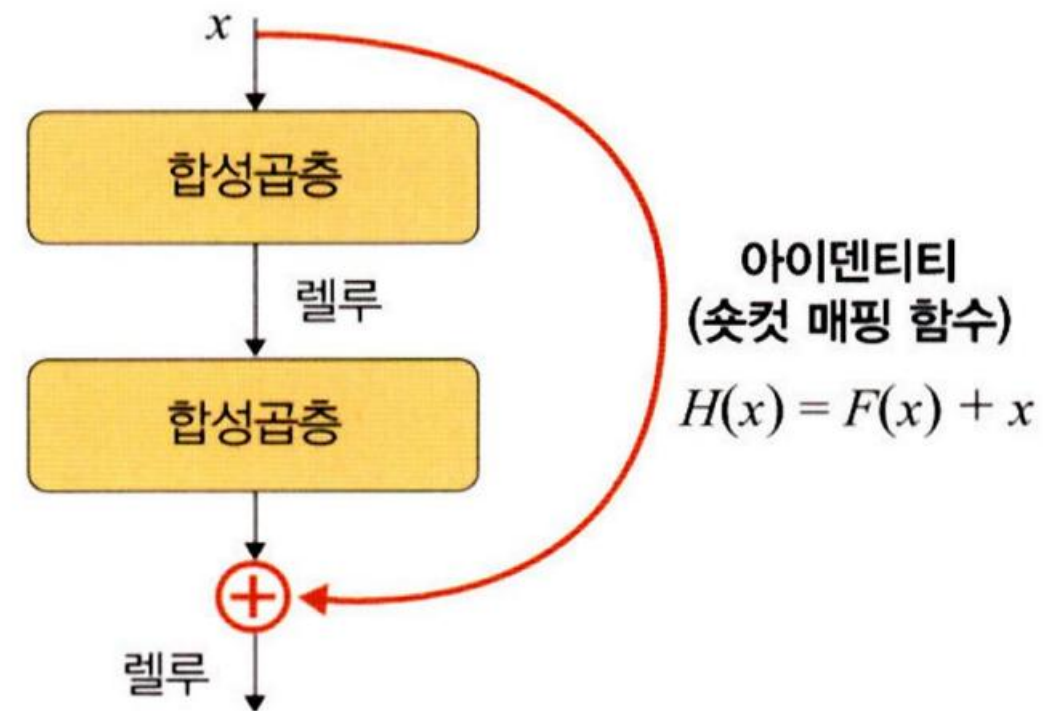
: 깊이에 따른 성능 저하 문제 해결 위해 레지듀얼 블록(Residual Block) 도입

⇒ 총 152개의 층으로 구성되어 기울기 소멸 문제가 발생할 수 있는 ResNet에서
기울기가 잘 전파되도록 숏컷 역할 수행

▼ 그림 6-25 ResNet 구조



기존 방식

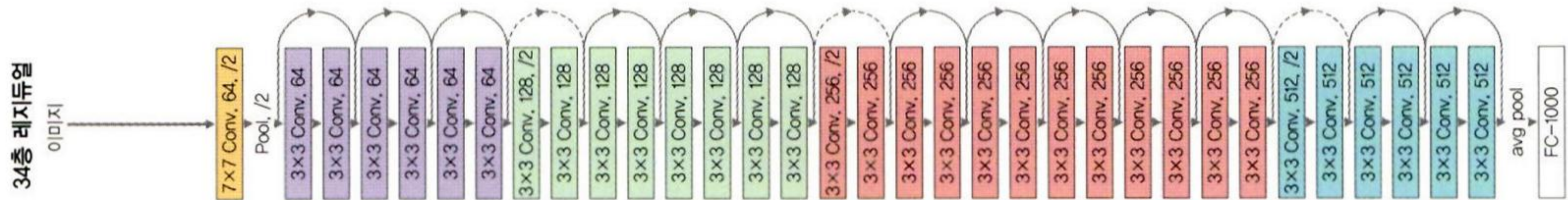


ResNet

#01 ResNet 구조의 주요 특징에 따른 문제점

- 블록: 하나로 묶인 계층들, 합성곱층을 하나의 블록으로 묶음.
- 레지듀얼 블록: 묶인 블록들 / ResNet: 레지듀얼 블록을 여러 개 쌓은 것

▼ 그림 6-26 ResNet 모델 전체 네트워크



=> 계층이 깊어질수록 파라미터 무제한으로 증가하는 문제 발생

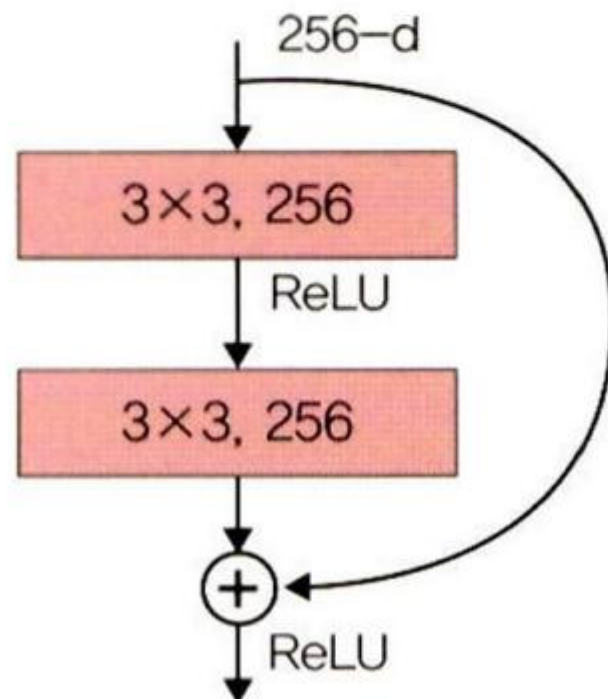
#01 병목 블록(Bottleneck Block)의 도입

- 예시: ResNET34 vs ResNET50

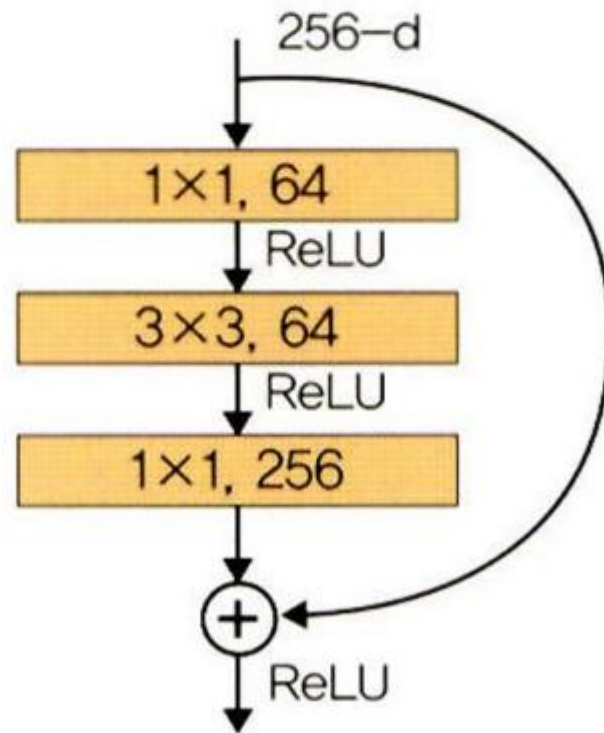
: ResNET50 쪽이 깊이가 깊음에도
파라미터 감소, 3x3 합성곱층 앞뒤로
1x1 합성곱층을 붙여 채널 수를 조정.
(차원을 줄였다 늘리기 가능해짐)

▼ 그림 6-27 기본 블록과 병목 블록

ResNet34
레지듀얼 블록



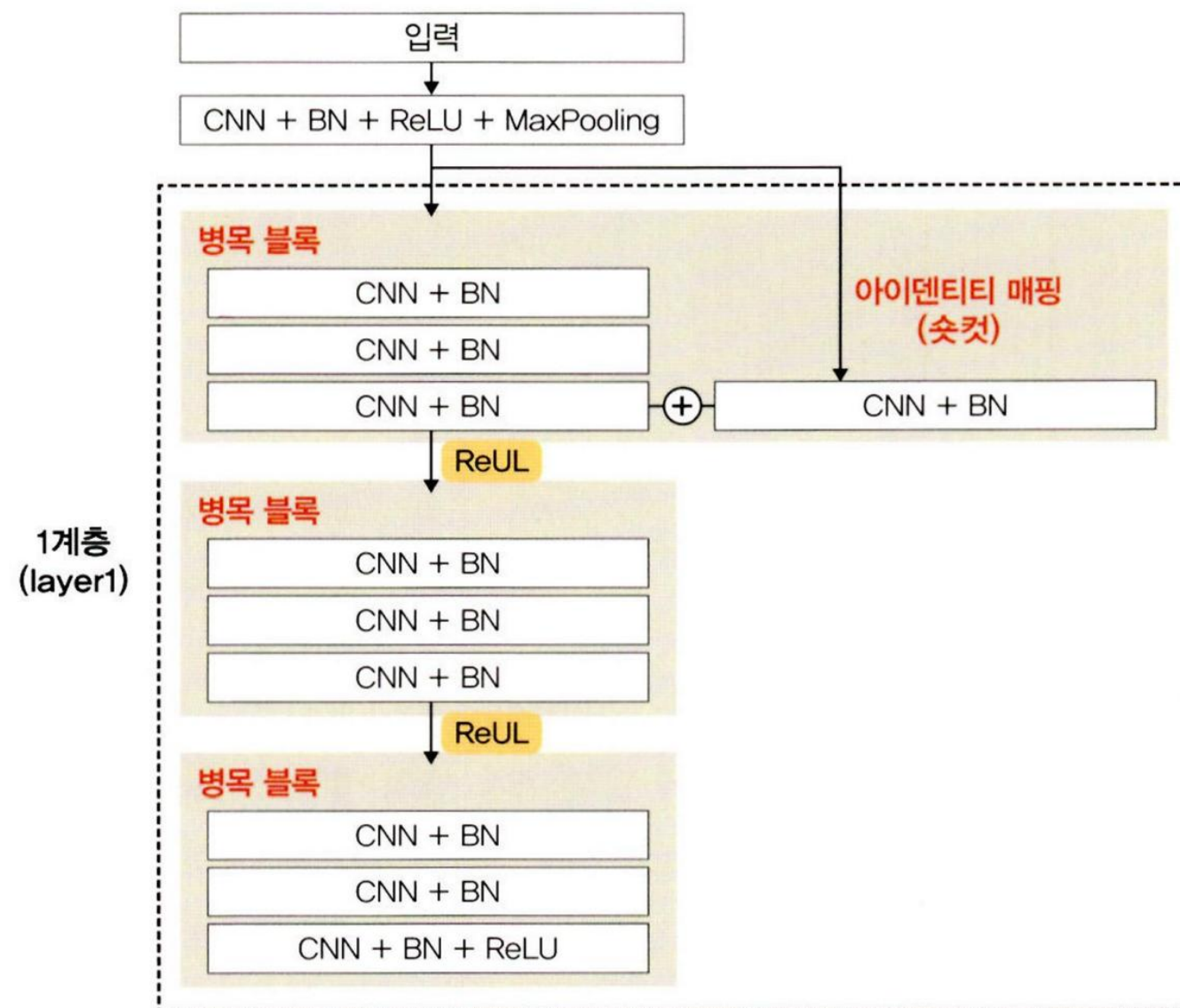
ResNet50
레지듀얼 블록



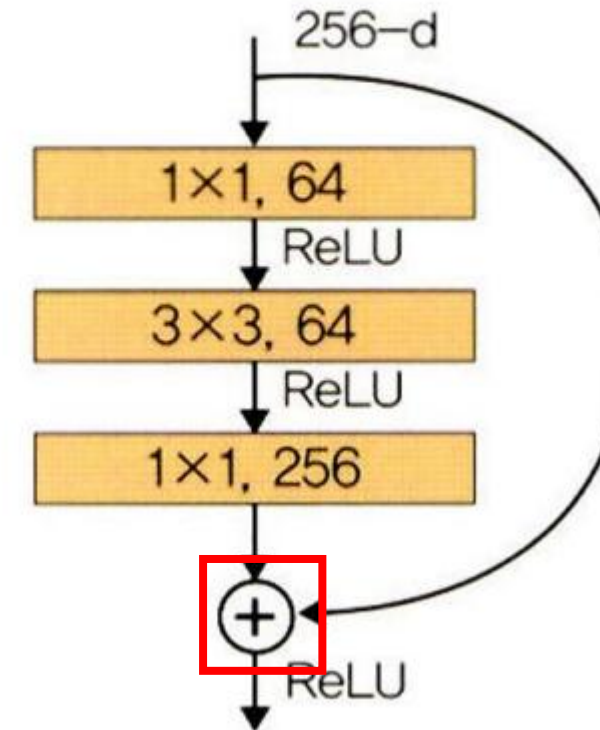
#01 아이덴티티 매핑(Shortcut) 설명

- 아이덴티티 매핑 = 숏컷 = 스킵 연결(skip connection)
: 입력 x 가 어떤 함수를 통과하더라도 x 라는 형태로 출력되게 하는 것.

▼ 그림 6-28 아이덴티티 매핑(숏컷)



ResNet50
레지듀얼 블록



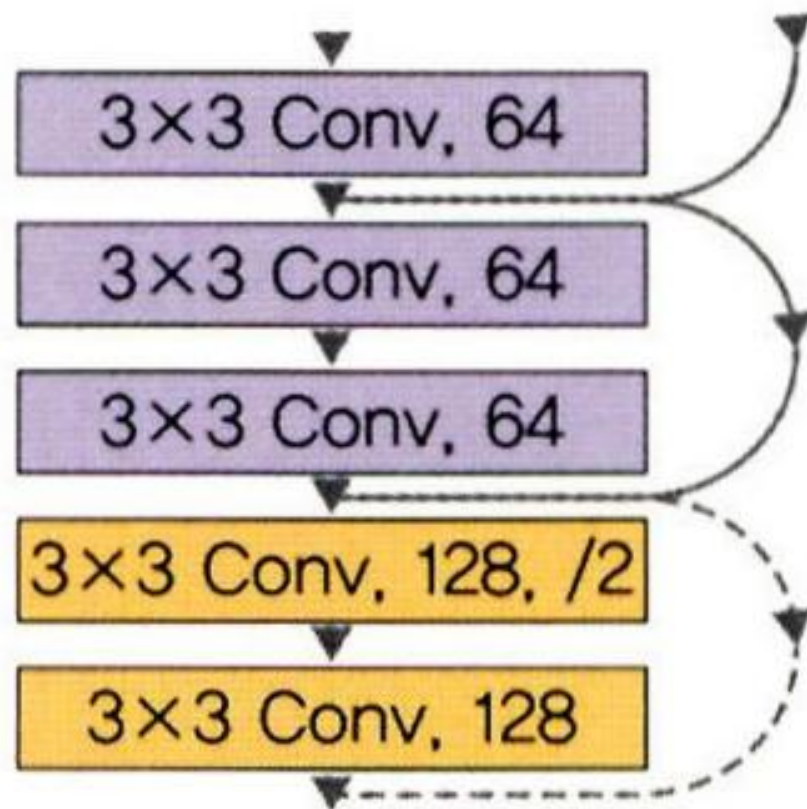
#01 아이덴티티 매핑(Shortcut) 설명

: 입력 x 를 i 에 저장했다가 합성곱층을 거친 x 에 i 를 더함으로써 형태 유지시킴

```
def forward(self, x):  
    i = x  
    x = self.conv1(x)  
    x = self.bn1(x)  
    x = self.relu(x)  
    x = self.conv2(x)  
    x = self.bn2(x)  
  
    if self.downsample is not None:  
        i = self.downsample(i) ----- 다운샘플 적용  
  
    x += i ----- 아이덴티티 매핑 적용  
    x = self.relu(x)  
    return x
```

#01 다운샘플링(Downsampling) 설명

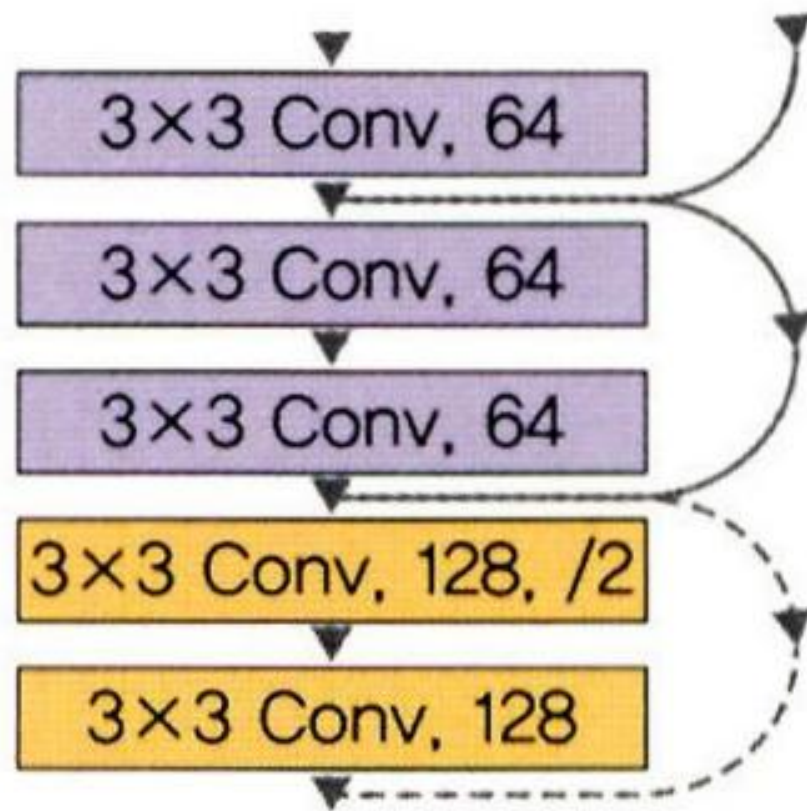
- 다운 샘플링: 특성맵 크기를 줄이기 위한 것으로, 풀링과 같은 역할.
: 보라색 영역에서 노란색 영역으로 넘어갈 때 다른 형태를 맞추기 위해(아이덴티티 매핑을 수행하기 위해) 아이덴티티에 대해 다운샘플 수행



프로젝션 숏컷/합성곱 블록
(입력 및 출력 차원 상이)
=> 아이덴티티 블록 (입출력 차원 동일)

#01 다운샘플링(Downsampling) 설명

- 다운 샘플링: 특성맵 크기를 줄이기 위한 것으로, 풀링과 같은 역할.
: 보라색 영역에서 노란색 영역으로 넘어갈 때 다른 형태를 맞추기 위해(아이덴티티 매핑을 수행하기 위해) 아이덴티티에 대해 다운샘플 수행



프로젝션 숏컷/합성곱 블록
(입력 및 출력 차원 상이)
⇒ 아이덴티티 블록 (입출력 차원 동일)

VGG19 구조 + 합성곱층(깊이 추가)
+ 숏컷을 추가해 기울기 소멸 문제 해결

#01 데이터 준비 및 전처리

코드 6-66 이미지 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), ----- 훈련 이미지 데이터에 대한 전처리
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]) ----- 검증과 테스트 이미지 데이터에 대한 전처리
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

사전 진행: 라이브러리 호출

코드 6-67 변수에 대한 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

#01 데이터 준비 및 전처리

코드 6-68 훈련과 테스트 데이터셋 불러오기

```
cat_directory = r'../chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'../chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_
    directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_
    directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]
```

코드 6-69 데이터셋을 훈련, 검증, 테스트 용도로 분리

```
random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
print(len(train_images_filepaths), len(val_images_filepaths),
    len(test_images_filepaths))
```

개, 고양이 예제 사용

훈련 400 / 검증 92
테스트 데이터셋 10개

#01 데이터 준비 및 전처리

코드 6-71 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size,
                                                                              mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size,
                                                                              mean, std), phase='val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

```
torch.Size([3, 224, 224])
0
```

=> index[0]의 이미지 정보 컬러(채널 3), 224x224, 0(고양이)

#01 데이터 준비 및 전처리

코드 6-72 데이터셋의 데이터를 메모리로 불러오기

```
train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_iterator, 'val': valid_iterator}

batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

데이터로더(DataLoader)를 이용해 훈련 데이터 셋의 이미지 크기와 레이블 출력

```
torch.Size([32, 3, 224, 224])
tensor([0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1,
0, 1, 0, 1, 0, 1])
```

02 ResNet (2)



#02 ResNet 구현

ResNet의 기본 블록: BasicBlock 구조

- ResNet18, ResNet34에서 사용되는 기본 블록
- 두 개의 3×3 합성곱(Convolution Layer) 으로 구성
- 입력값을 그대로 더하는 잔차 연결(Residual Connection) 포함
- 입력과 출력의 크기가 다를 경우 다운샘플링(Downsampling) 적용

```
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride = 1, downsample = False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size = 3,
                                stride = stride, padding = 1, bias = False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size = 3,
                                stride = 1, padding = 1, bias = False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace = True)
```


#02 ResNet 구현

Downsampling

- 입력과 출력의 크기가 다를 때 크기를 맞춰주는 과정
- stride=2 설정을 통해 특성 맵 크기 절반으로 축소
- 1×1 합성곱(conv)으로 채널 수 조정

```
if downsample:
    conv = nn.Conv2d(in_channels, out_channels, kernel_size = 1,
                     stride = stride, bias = False)
    bn = nn.BatchNorm2d(out_channels)
    downsample = nn.Sequential(conv, bn)
else:
    downsample = None
self.downsample = downsample
```

#02 ResNet 구현

스킵 연결

- 입력값을 그대로 더하는 스킵 연결(skip connection)
- 출력 = $F(x) + x$ 형태로 계산
- 모델이 “변화량(Residual)”만 학습하도록 함
- 깊은 신경망에서 학습 안정성 향상

```
def forward(self, x):  
    i = x  
    x = self.conv1(x)  
    x = self.bn1(x)  
    x = self.relu(x)  
    x = self.conv2(x)  
    x = self.bn2(x)  
  
    if self.downsample is not None:  
        i = self.downsample(i)  
  
    x += i  
    x = self.relu(x)  
  
    return x
```

#02 ResNet 구현

Bottleneck 블록 구조

- Bottleneck 블록은 ResNet50, ResNet101, ResNet152에서 사용
- 3개의 합성곱 계층으로 구성
 - 1×1 합성곱 → 채널 수 축소
 - 3×3 합성곱 → 특징 추출
 - 1×1 합성곱 → 채널 수 확장
- $\text{expansion} = 4$ (출력 채널 수가 입력 채널의 4배로 늘어남)

```
class Bottleneck(nn.Module):
    expansion = 4

    def __init__(self, in_channels, out_channels, stride = 1, downsample = False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size = 1, stride = 1, bias = False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size = 3, stride = stride, padding = 1, bias = False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion * out_channels, kernel_size = 1,
                                stride = 1, bias = False)
        self.bn3 = nn.BatchNorm2d(self.expansion * out_channels)
        self.relu = nn.ReLU(inplace = True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion * out_channels, kernel_size = 1,
                              stride = stride, bias = False)
            bn = nn.BatchNorm2d(self.expansion * out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample
```

Bottleneck 블록의 출력 채널 수 =
 $\text{expansion} \times \text{out_channels}$

#02 ResNet 구현

Bottleneck 블록의 특징과 장점

- 기본 블록(BasicBlock)보다 더 깊은 계층 쌓기가 가능
- 계산량을 줄이면서도 비선형성(Non-linearity) 을 많이 포함
- 다양한 입력 데이터에 대한 복잡한 표현 학습 가능
- ResNet을 더 깊게 설계할 수 있도록 도움
- 1×1 (축소) $\rightarrow 3 \times 3$ (처리) $\rightarrow 1 \times 1$ (확장)

#02 ResNet 구현

ResNet 네트워크 구조

- ResNet은 VGGNet처럼 블록 반복 구조를 가짐
- config를 통해 블록 수, 채널 수 설정
- 주요 구성 요소:
 - 초기 합성곱 + BatchNorm + MaxPooling
 - layer1 ~ layer4: 잔차 블록(Residual Block) 계층
 - Global Average Pooling + Fully Connected Layer
- 다양한 모델 (ResNet18~152)은 이 구조 기반으로 블록 수만 다름

```
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()

        block, n_blocks, channels = config
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size = 7, stride = 2, padding = 3, bias = False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace = True)
        self.maxpool = nn.MaxPool2d(kernel_size = 3, stride = 2, padding = 1)

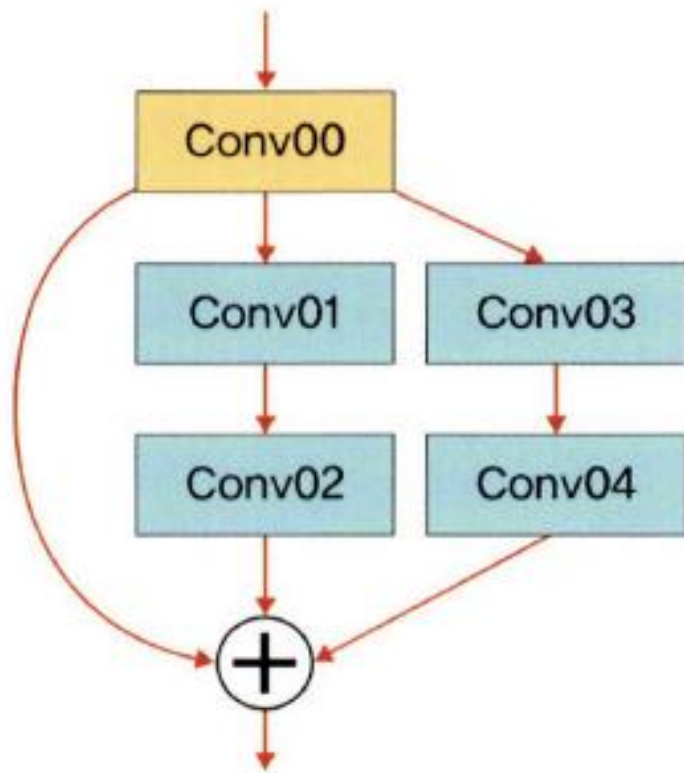
        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride = 2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride = 2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride = 2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(self.in_channels, output_dim)
```

#02 ResNet 구현

Residual Branch의 BN=0 초기화

- 각 잔차 분기(residual branch) 의 마지막 BN(γ , weight) 를 0으로 초기화
→ 첫 학습 단계에서 해당 분기는 identity매핑 으로 시작
- 효과: 학습 안정성 ↑, 수렴 가속, Top-1 성능 약 0.2~0.3% 향상



```
if zero_init_residual:
    for m in self.modules():
        if isinstance(m, Bottleneck):
            nn.init.constant_(m.bn3.weight, 0)
        elif isinstance(m, BasicBlock):
            nn.init.constant_(m.bn2.weight, 0)
```

#02 ResNet 구현

get_resnet_layer() 함수

- 각 layer는 여러 개의 Residual Block으로 구성
- 입력 채널과 출력 채널이 다르면 downsample=True
→ 1 × 1 합성곱으로 크기 조정
- 반복문으로 블록을 n_blocks만큼 추가
- 마지막에 Sequential 형태로 반환

```
def get_resnet_layer(self, block, n_blocks, channels, stride = 1):  
    layers = []  
    if self.in_channels != block.expansion * channels:  
        downsample = True  
    else:  
        downsample = False  
  
    layers.append(block(self.in_channels, channels, stride, downsample))  
  
    for i in range(1, n_blocks):  
        layers.append(block(block.expansion * channels, channels))  
  
    self.in_channels = block.expansion * channels  
    return nn.Sequential(*layers)
```

#02 ResNet 구현

```
def forward(self, x):  
    x = self.conv1(x)  
    x = self.bn1(x)  
    x = self.relu(x)  
    x = self.maxpool(x)  
    x = self.layer1(x)  
    x = self.layer2(x)  
    x = self.layer3(x)  
    x = self.layer4(x)  
    x = self.avgpool(x)  
    h = x.view(x.shape[0], -1)  
    x = self.fc(h)  
    return x, h
```

입력 이미지: (224 × 224)

각 단계에서의 feature map 크기 변화:

Conv → 224 × 224

MaxPool → 112 × 112

Layer1 → 56 × 56

Layer2 → 28 × 28

Layer3 → 14 × 14

Layer4 → 7 × 7

#02 ResNet 구현

ResNetConfig

- ResNet은 다양한 깊이(ResNet18~152)를 지원
- 각 모델은 기본 블록 구조, 블록 수, 채널 수가 다름
- 이를 쉽게 관리하기 위해 ResNetConfig(namedtuple) 사용

```
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])
```

- 구성 요소
 - block : 사용할 블록 종류 (BasicBlock / Bottleneck)
 - n_blocks : 각 계층(layer1~4)에 들어갈 블록 개수
 - channels : 각 계층의 출력 채널 수

#02 ResNet 구현

BasicBlock vs Bottleneck 구성

- 기본 블록 (BasicBlock) 사용 - $\rightarrow$ 3×3 합성곱 2개로 구성, 얇지만 계산 효율적

```
resnet18_config = ResNetConfig(block = BasicBlock,  
                                n_blocks = [2,2,2,2],  
                                channels = [64, 128, 256, 512])  
  
resnet34_config = ResNetConfig(block = BasicBlock,  
                                n_blocks = [3,4,6,3],  
                                channels = [64, 128, 256, 512])
```

- 병목 블록 (Bottleneck) 사용 $\rightarrow 1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ 구조, 깊고 성능이 높음

```
resnet50_config = ResNetConfig(block = Bottleneck,  
                                n_blocks = [3, 4, 6, 3],  
                                channels = [64, 128, 256, 512])  
  
resnet101_config = ResNetConfig(block = Bottleneck,  
                                 n_blocks = [3, 4, 23, 3],  
                                 channels = [64, 128, 256, 512])  
  
resnet152_config = ResNetConfig(block = Bottleneck,  
                                 n_blocks = [3, 8, 36, 3],  
                                 channels = [64, 128, 256, 512])
```

Add code cell
⌘/Ctrl+M B

#02 ResNet 구현

사전 학습된 ResNet 모델 사용

- 복잡한 블록 정의 없이 한 줄 코드로 ResNet 사용 가능
- torchvision.models 에서 사전 학습된 가중치 포함 모델 제공
- 학습 시간 절약 + 높은 성능 확보

```
pretrained_model = models.resnet50(pretrained = True)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)
```

#02 ResNet 구현

직접 구현한 ResNet

```
OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)
```


#02 ResNet 구현

옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
```

- $lr=1e-7 \rightarrow$ 학습률(learning rate) = 10^{-7}
- CrossEntropyLoss $\rightarrow$ 분류(Classification) 문제에서 가장 자주 사용

모델 정확도 계산 함수 정의

```
def calculate_topk_accuracy(y_pred, y, k = 2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1)
        top_pred = top_pred.t()
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim = True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim = True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

- 모델이 얼마나 잘 예측했는지 확인하기 위한 함수
- Top-1, Top-2 정확도 계산 가능
- torch.topk() $\rightarrow$ 가장 높은 확률의 예측 인덱스를 반환
- .t() $\rightarrow$ 텐서 전치
- torch.eq() $\rightarrow$ 예측값과 실제값이 같은지 비교 (True/False 반환)

#02 ResNet 구현

모델 학습 함수 정의

- 한 epoch 동안 학습 데이터를 이용해 모델을 학습시키는 함수
- 손실(loss)과 정확도(top-1, top-5)를 계산

```
def train(model, iterator, optimizer, criterion, device):  
    epoch_loss = 0  
    epoch_acc_1 = 0  
    epoch_acc_5 = 0  
  
    model.train()  
    for (x, y) in iterator:  
        x = x.to(device)  
        y = y.to(device)  
  
        optimizer.zero_grad()  
        y_pred = model(x)  
  
        loss = criterion(y_pred[0], y)  
  
        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)  
        loss.backward()  
        optimizer.step()  
  
        epoch_loss += loss.item()  
        epoch_acc_1 += acc_1.item()  
        epoch_acc_5 += acc_5.item()  
  
    epoch_loss /= len(iterator)  
    epoch_acc_1 /= len(iterator)  
    epoch_acc_5 /= len(iterator)  
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

- model.train() → 드롭아웃, 배치정규화 활성화
- optimizer.zero_grad() → 누적된 gradient 초기화
- loss.backward() → 역전파 수행
- optimizer.step() → 파라미터 업데이트
- calculate_topk_accuracy() → top-1, top-5 정확도 계산

#02 ResNet 구현

모델 평가 함수 & 학습 시간 측정

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)
            acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)

            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

- 한 epoch 학습에 소요된 시간을 분(min), 초(sec) 단위로 반환

- model.eval() → 드롭아웃/배치정규화 비활성화
- torch.no_grad() → 메모리 절약, 속도 향상
- calculate_topk_accuracy() → top-1, top-5 정확도 계산

#02 ResNet 구현

모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc_1, train_acc_5 = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '/content/drive/MyDrive/EuronDL/chap06/ResNet-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc @1: {train_acc_1*100:6.2f}% | ' \
          f'\tTrain Acc @5: {train_acc_5*100:6.2f}%')
    print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc @1: {valid_acc_1*100:6.2f}% | ' \
          f'\tValid Acc @5: {valid_acc_5*100:6.2f}%')
```

- 손실값(Loss)은 약 0.7 부근에서 안정
- 정확도(Accuracy)는 약 50% 수준
- 데이터 수가 적고 학습 epoch이 짧기 때문에 높은 성능은 기대하기 어려움

```
Epoch: 01 | Epoch Time: 0m 9s
    Train Loss: 0.730 | Train Acc @1: 51.44% | Train Acc @5: 100.00%
    Valid Loss: 0.748 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 02 | Epoch Time: 0m 8s
    Train Loss: 0.727 | Train Acc @1: 50.72% | Train Acc @5: 100.00%
    Valid Loss: 0.743 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 03 | Epoch Time: 0m 7s
    Train Loss: 0.726 | Train Acc @1: 50.24% | Train Acc @5: 100.00%
    Valid Loss: 0.727 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 04 | Epoch Time: 0m 8s
    Train Loss: 0.728 | Train Acc @1: 50.00% | Train Acc @5: 100.00%
    Valid Loss: 0.725 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 05 | Epoch Time: 0m 11s
    Train Loss: 0.718 | Train Acc @1: 52.64% | Train Acc @5: 100.00%
    Valid Loss: 0.727 | Valid Acc @1: 50.15% | Valid Acc @5: 100.00%
Epoch: 06 | Epoch Time: 0m 7s
    Train Loss: 0.728 | Train Acc @1: 49.52% | Train Acc @5: 100.00%
    Valid Loss: 0.725 | Valid Acc @1: 50.15% | Valid Acc @5: 100.00%
Epoch: 07 | Epoch Time: 0m 8s
    Train Loss: 0.718 | Train Acc @1: 50.48% | Train Acc @5: 100.00%
    Valid Loss: 0.714 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 08 | Epoch Time: 0m 12s
    Train Loss: 0.715 | Train Acc @1: 50.24% | Train Acc @5: 100.00%
    Valid Loss: 0.712 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 09 | Epoch Time: 0m 7s
    Train Loss: 0.711 | Train Acc @1: 50.72% | Train Acc @5: 100.00%
    Valid Loss: 0.712 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
Epoch: 10 | Epoch Time: 0m 11s
    Train Loss: 0.704 | Train Acc @1: 50.24% | Train Acc @5: 100.00%
    Valid Loss: 0.708 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00%
```

#02 ResNet 구현

테스트 데이터 예측 및 결과 저장

```
import pandas as pd
id_list = []
pred_list = []
_id=0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id=test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[:, 1].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('/content/drive/MyDrive/EuronDL/chap06/ReNet.csv', index=False)
res.head(10)
```

	id	label
0	109	0.604981
1	145	0.651012
2	15	0.601149
3	162	0.645796
4	167	0.594723
5	200	0.648900
6	210	0.614105
7	211	0.640876
8	213	0.621526
9	224	0.586611

- label 값이 0.5보다 크면 → 개
- label 값이 0.5보다 작으면 → 고양이

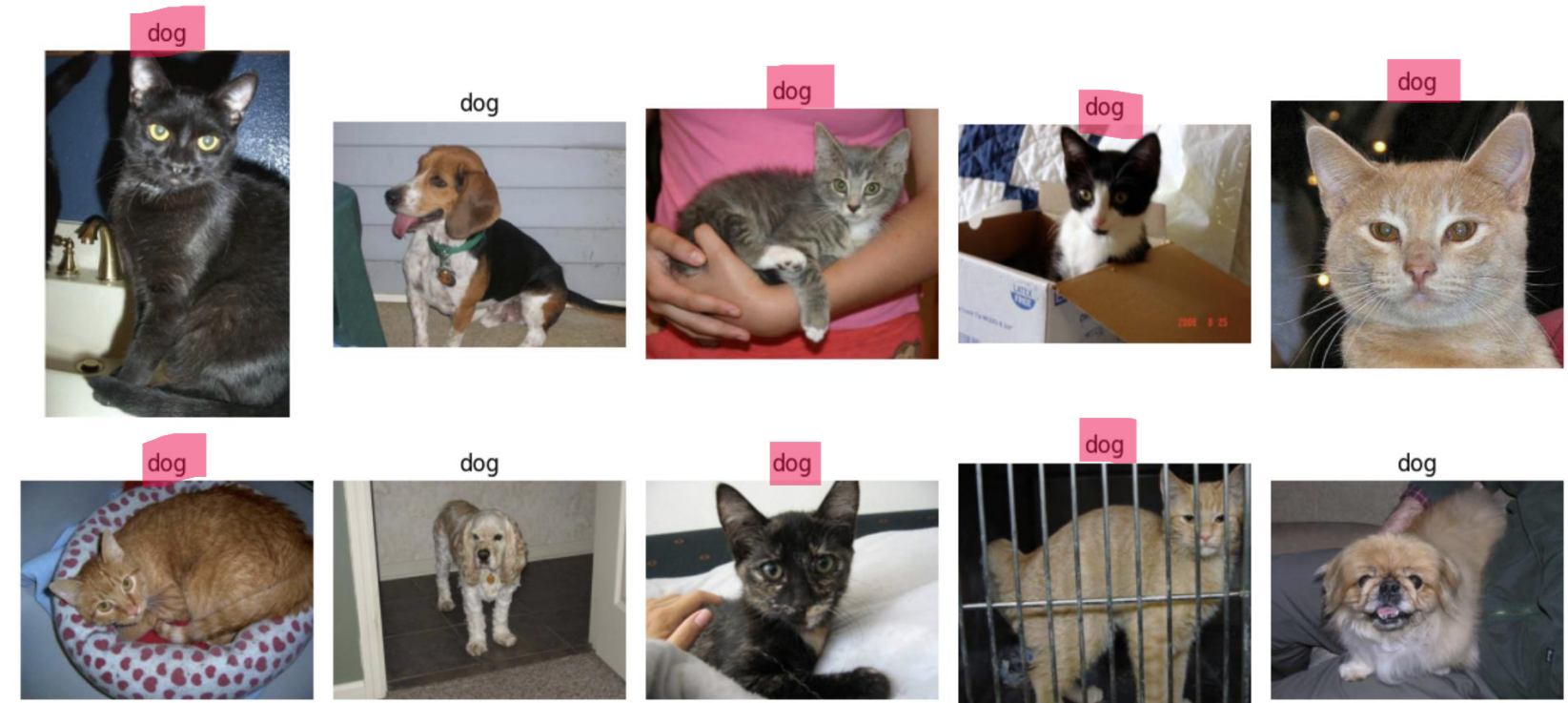
#02 ResNet 구현

테스트 이미지 예측 결과 시각화

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
display_image_grid(test_images_filepaths)
```



03-1 객체 인식을 위한 신경망



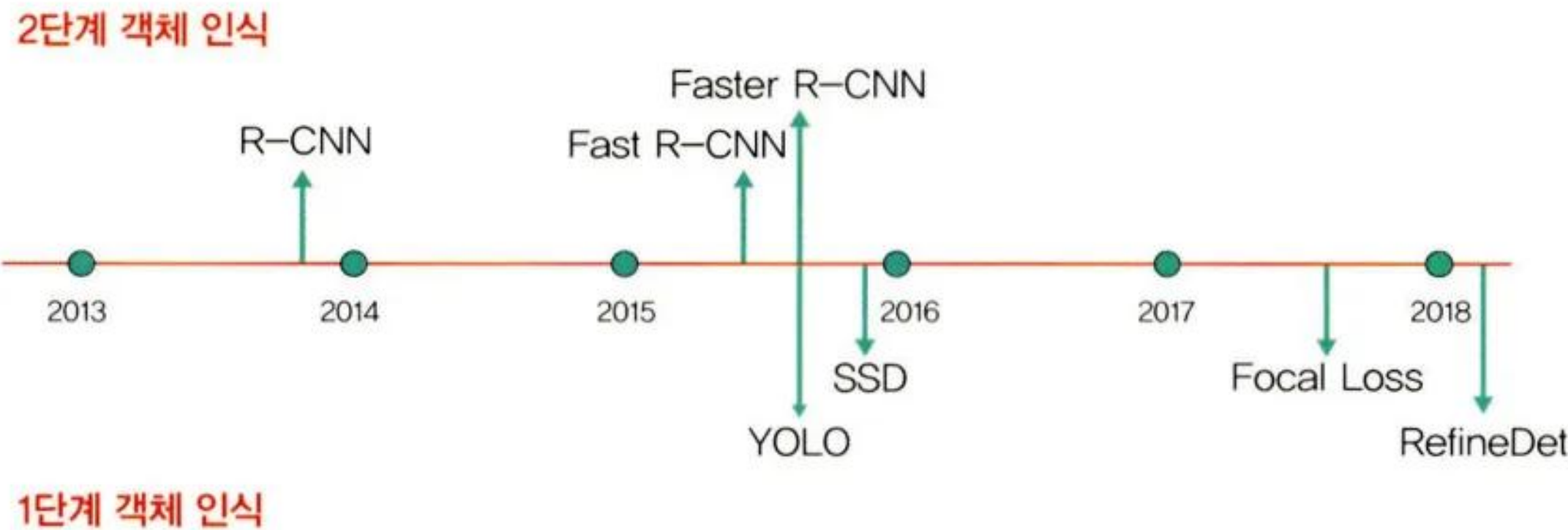
#03-1 객체 인식(Object Detection) 개요

- 이미지 속 '무엇(What)' 과 '어디(Where)' 를 동시에 파악
- 분류(Classification) + 위치 검출(Localization)
- 출력: 각 객체의 클래스 이름 + 바운딩 박스 좌표

#03-1 1단계 vs 2단계 객체 인식

구분	방식	속도	정확도	대표모델
1단계 (1-stage)	분류와 검출을 동시에 수행	빠름	낮음	YOLO, SSD
2단계 (2-stage)	후보 영역-> 분 류 순서로 수행	느림	높음	R-CNN계열

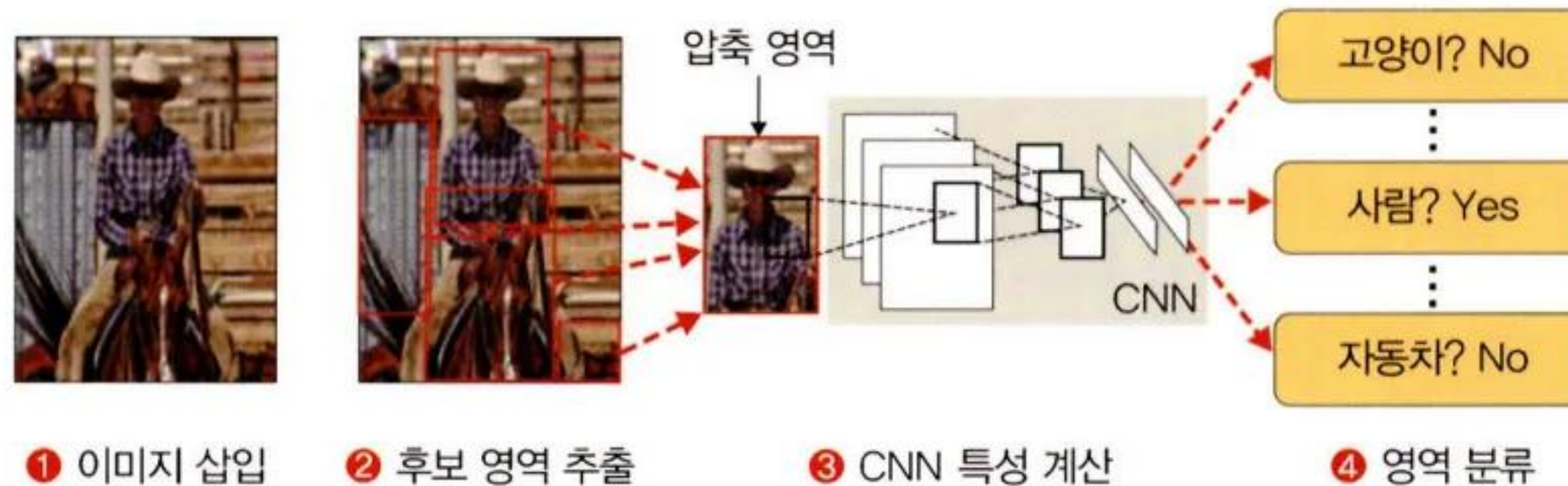
▼ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도



#03-1 R-CNN 구조

- CNN + 후보 영역(region proposal) 알고리즘 결합
- 선택적 탐색(Selective Search)으로 후보 영역 약 2000개 생성
- 각 영역을 CNN에 넣어 분류
- 한계: 세 단계의 복잡한 과정, 긴 학습시간, 속도 문제

▼ 그림 6-36 R-CNN 학습 절차



#03-1 선택적 탐색(Selective Search) 과정

1. 초기 영역 생성 → 다수의 작은 영역으로 분할



입력 이미지



분할



후보 영역

2. 비슷한 영역끼리 통합



입력 이미지



초기 분할



비슷한 영역 통합



비슷한 영역 통합

#03-1 선택적 탐색(Selective Search) 과정

3. 후보 영역 생성 (bounding box 추출)



입력 이미지

비슷한 영역을 통합하여 후보 영역 생성

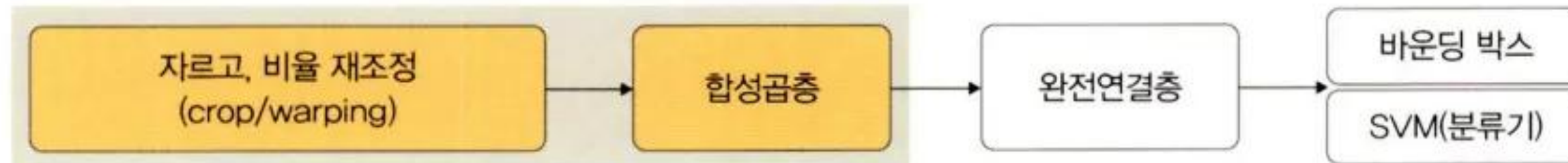
-> 객체가 있을 법한 영역을 효율적으로 찾음

#03-1 공간 피라미드 풀링 (SPPNet)

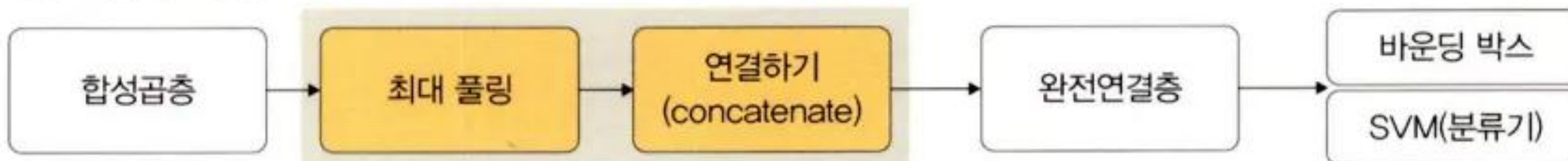
- 기존 CNN은 입력 크기를 고정해야 함 → cropping 필요
- SPPNet은 입력 크기와 상관없이 합성곱 가능
- 완전연결층에 전달되기 전, 특성맵 크기를 통일

▼ 그림 6-40 공간 피라미드 풀링

R-CNN



공간 피라미드 풀링

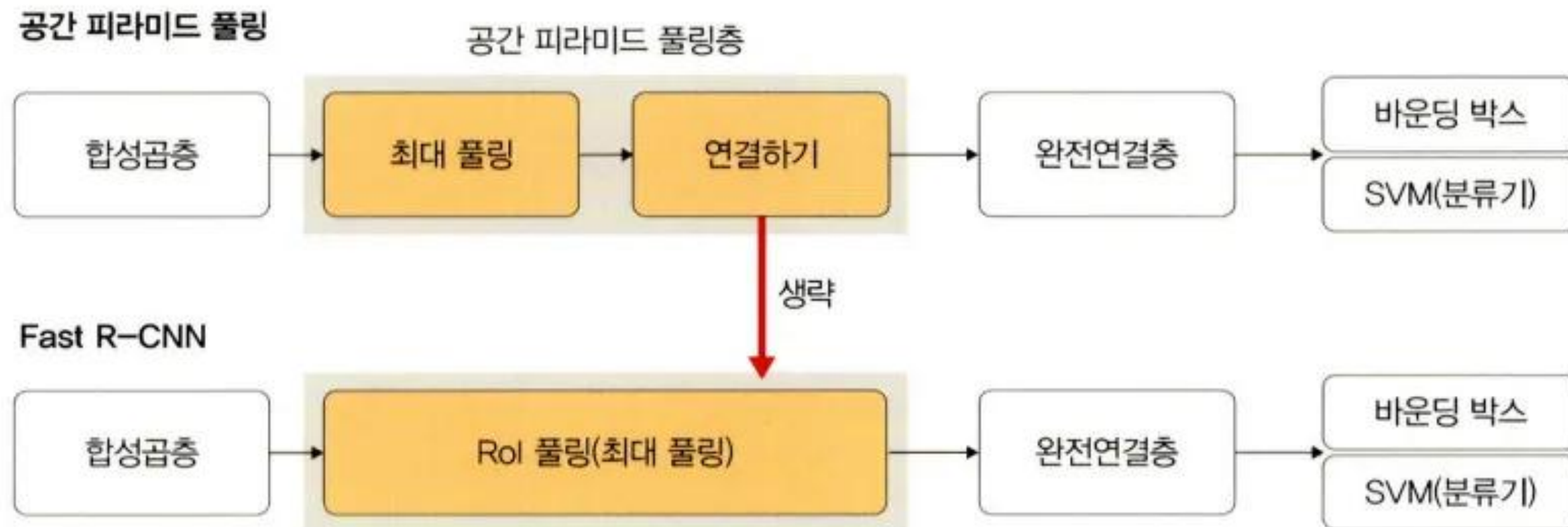


공간 피라미드 풀링층

#03-1 Fast R-CNN

- R-CNN의 속도 문제 해결
- 바운딩 박스마다 CNN 적용 대신 하나의 CNN으로 전체 이미지를 처리하고 RoI Pooling 적용
- 학습 시간 단축 및 효율 향상

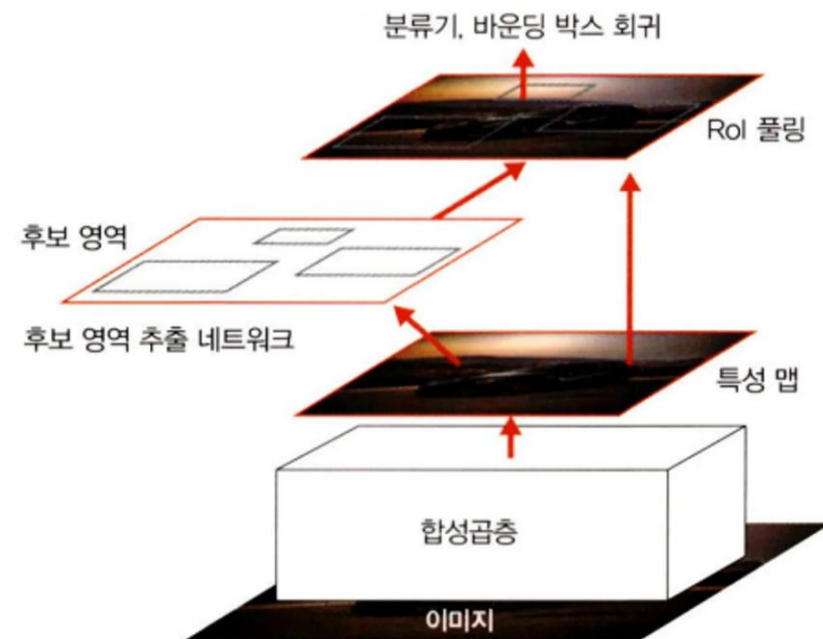
▼ 그림 6-41 Fast R-CNN



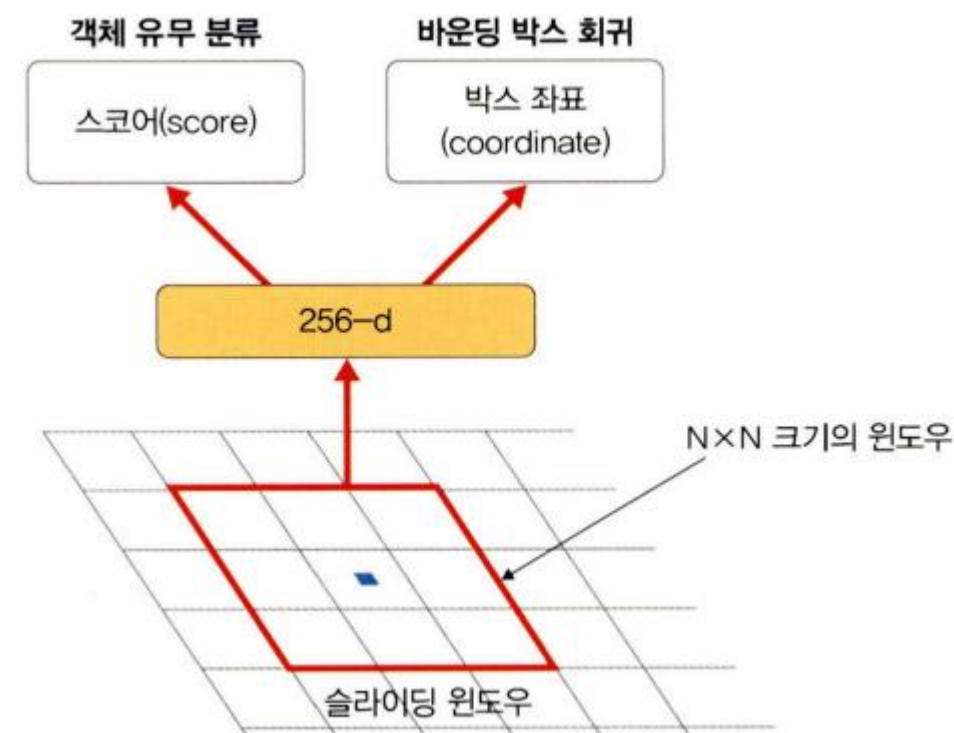
#03-1 Faster R-CNN & RPN

- 후보 영역 생성도 CNN 내부에서 수행 → RPN (Region Proposal Network)
- RPN이 각 위치에서 객체 존재 여부와 바운딩 박스 좌표 예측
- 앵커(anchor) 방식을 통해 다양한 크기 · 비율의 객체 탐지 가능

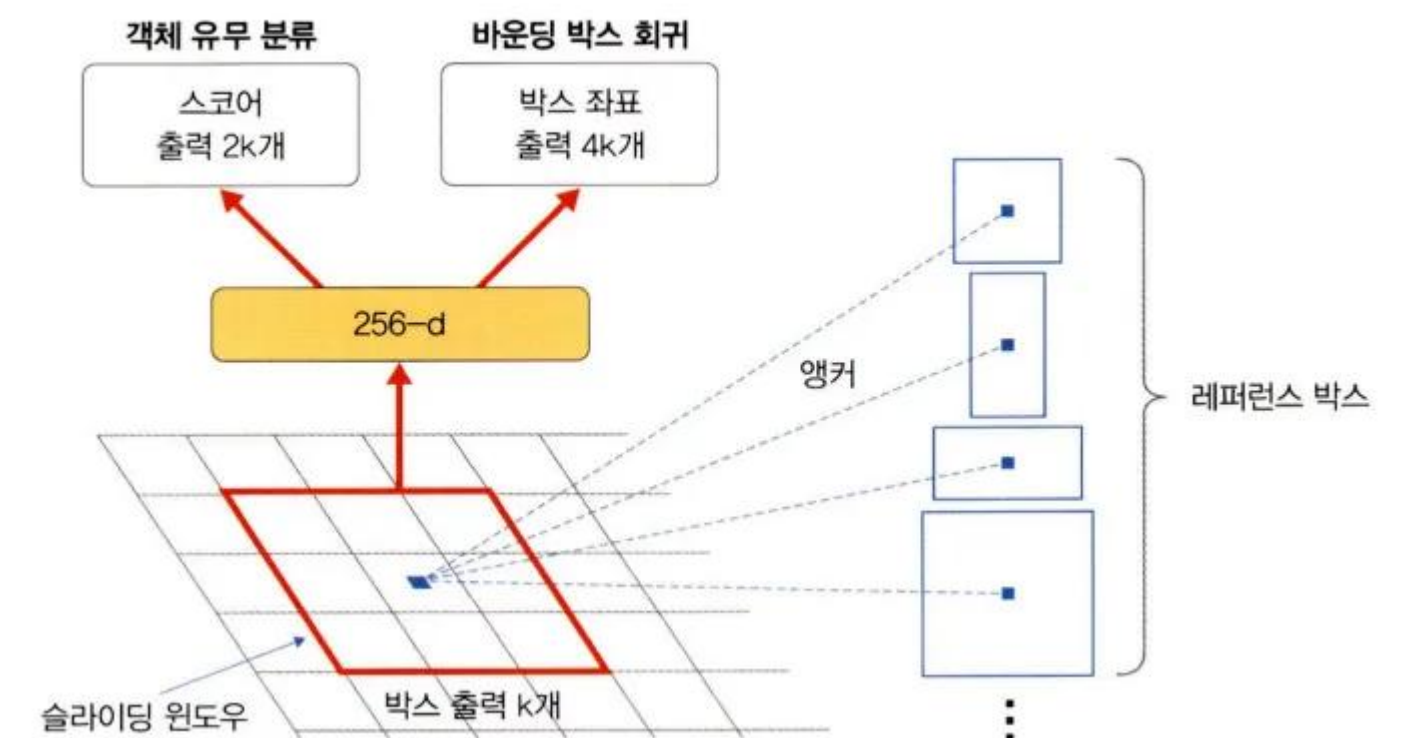
▼ 그림 6-43 Faster R-CNN



▼ 그림 6-44 후보 영역 추출 네트워크



▼ 그림 6-45 앵커



03-2 이미지 분할을 위한 신경망



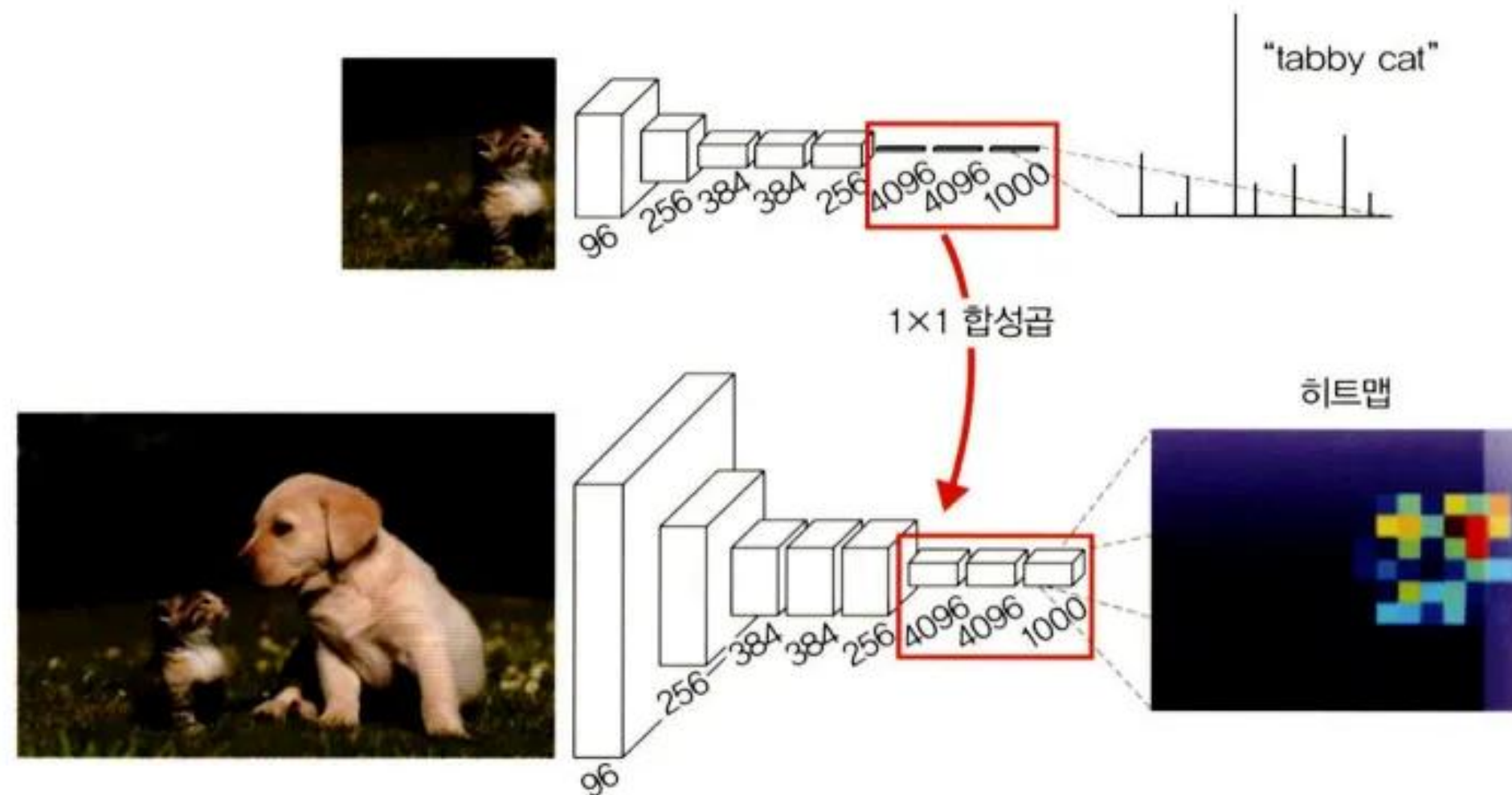
#03-2 이미지 분할(Image Segmentation) 개요

- 픽셀 단위로 객체를 구분
- 객체마다 색상이 다른 마스크(mask) 생성
- 응용: 의료 영상, 자율주행, 위성사진 분석

#03-2 완전 합성곱 네트워크 (FCN)

- 완전연결층 대신 1x1 합성곱 사용 → 위치 정보 보존
- 입력 크기 제약 없음
- CNN 기반 분류 모델을 변형하여 픽셀 수준 분할 수행

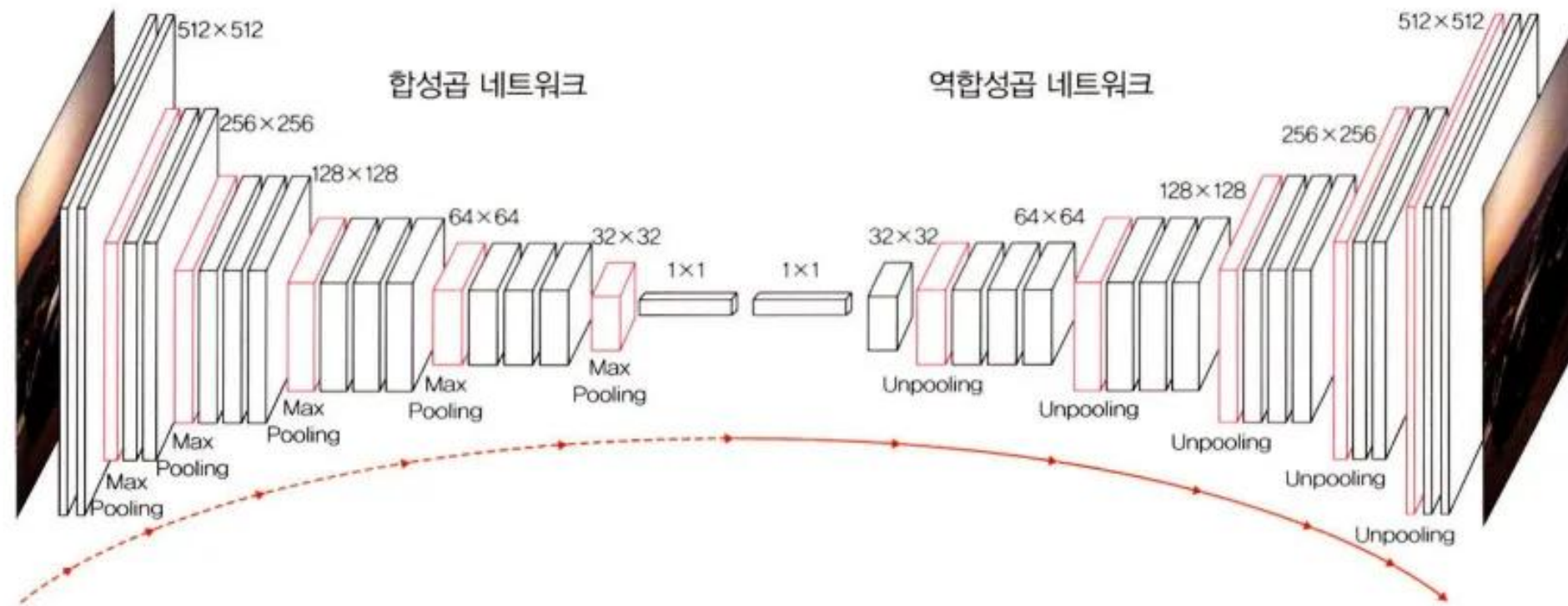
▼ 그림 6-46 완전 합성곱 네트워크



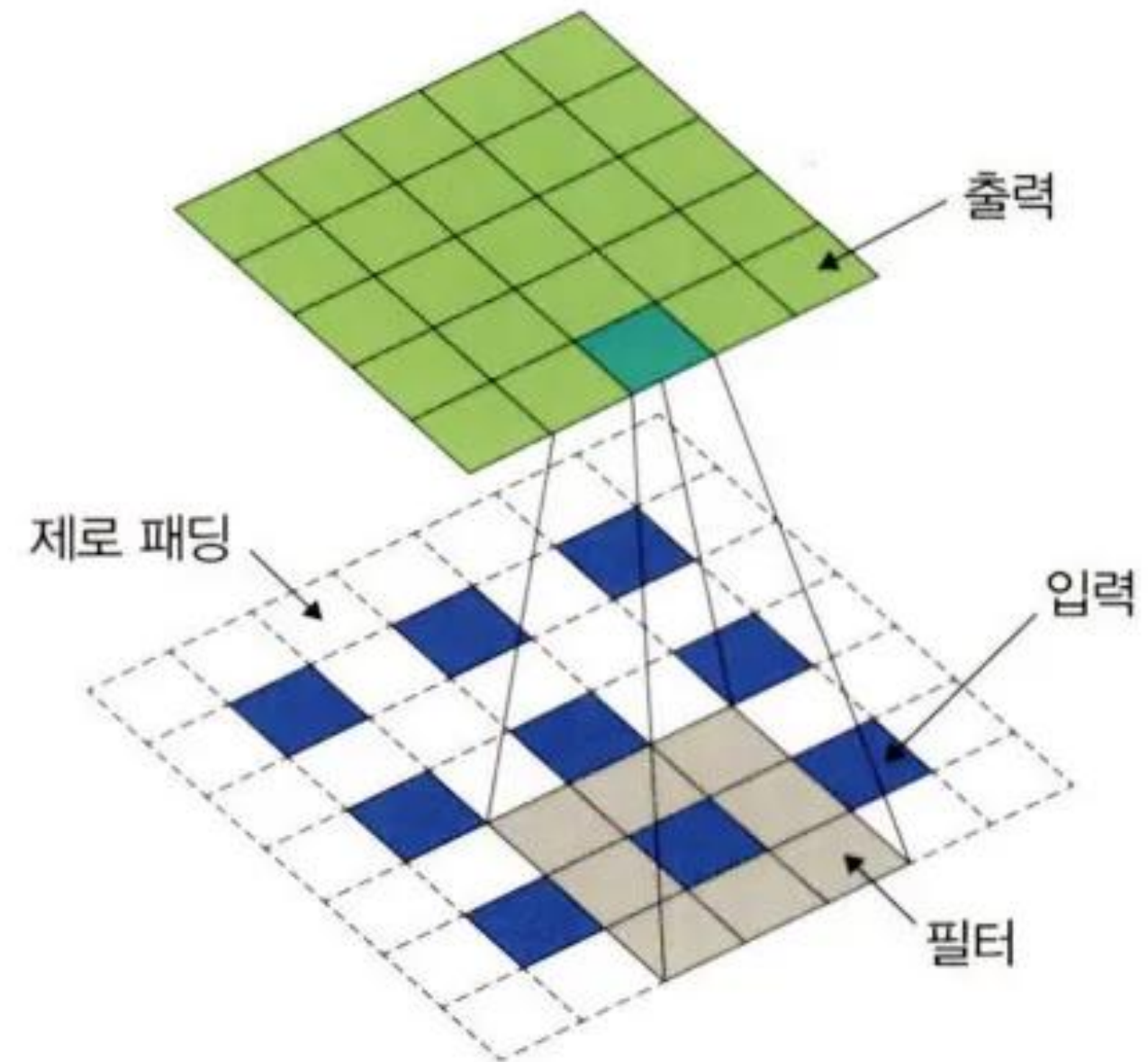
#03-2 합성곱 & 역합성곱 네트워크

- 해상도 저하 문제를 해결하기 위해 역합성곱(Deconvolution) 도입
- 업샘플링을 통해 입력 크기 복원
- 시멘틱 분할(semantic segmentation)에 활용

▼ 그림 6-47 합성곱 & 역합성곱 네트워크



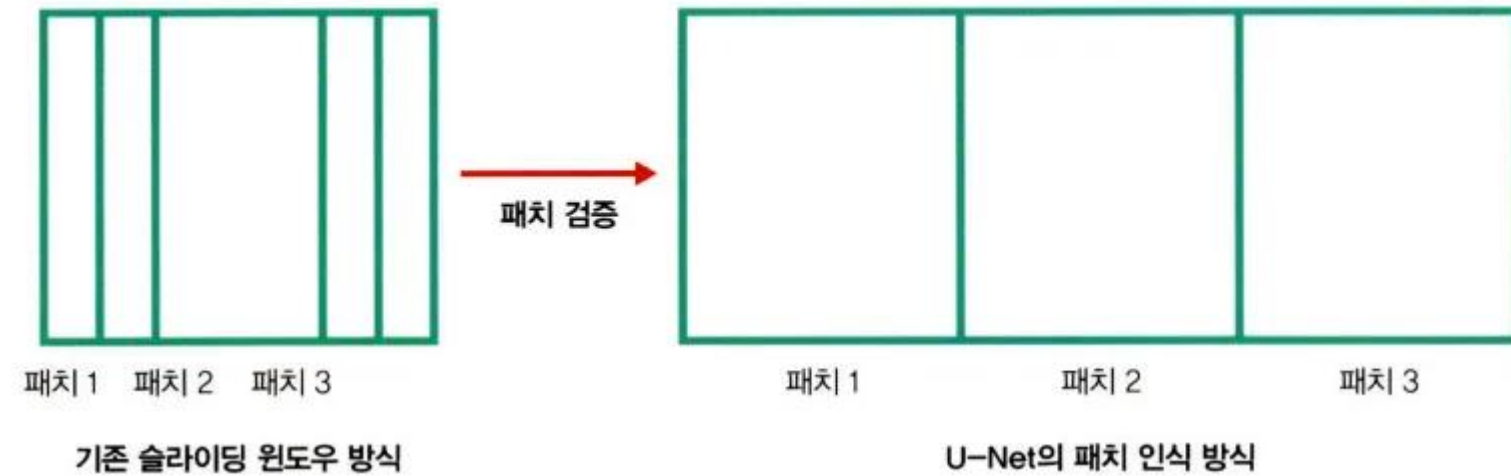
▼ 그림 6-48 역합성곱 진행 방식



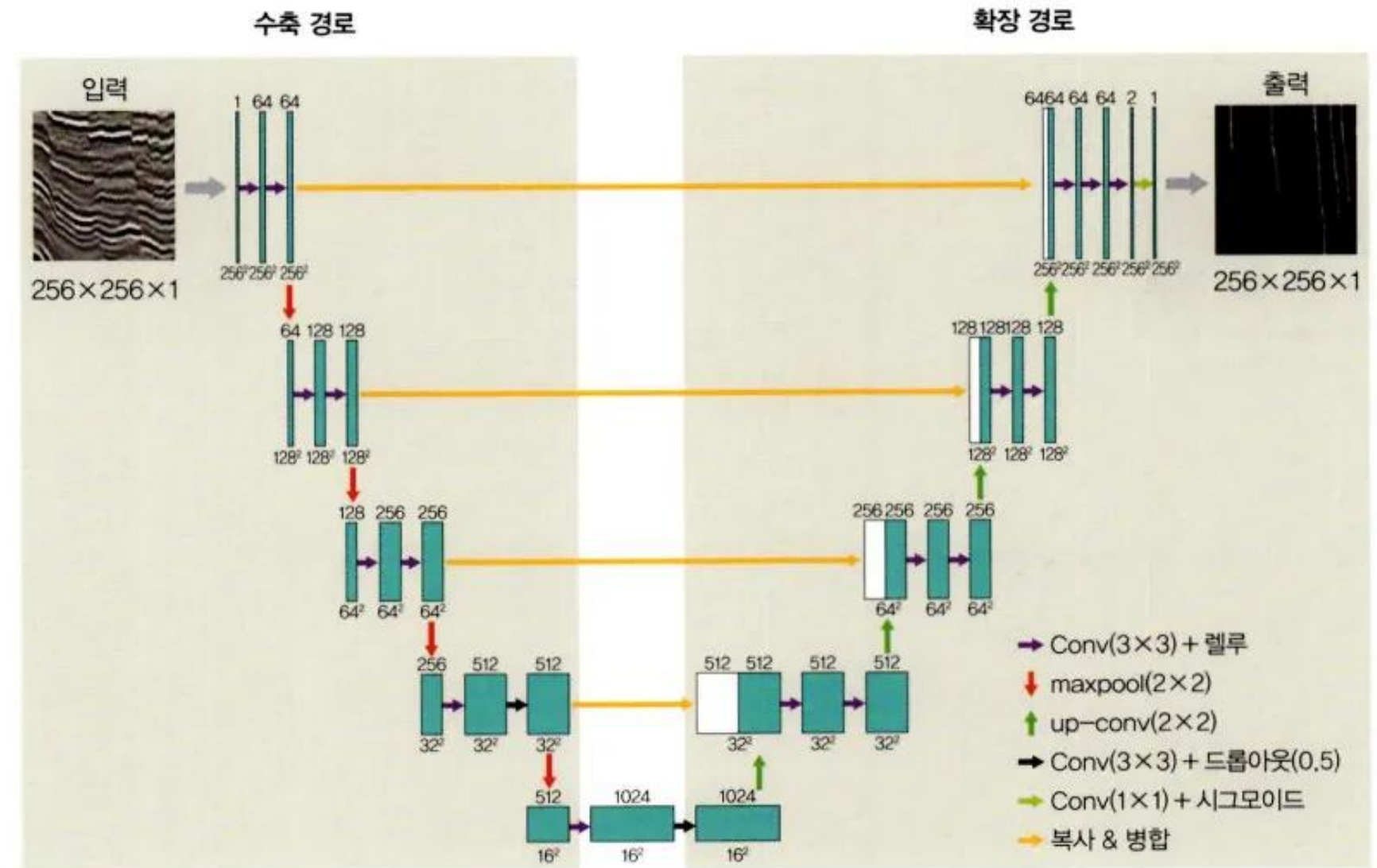
#03-2 U-Net

- 바이오 메디컬 영상 분할용 모델
- 인코더-디코더 구조 (수축 경로 + 확장 경로)
- 컨텍스트 인식 + 지역화 정확도 향상

▼ 그림 6-49 슬라이딩 윈도우 방식



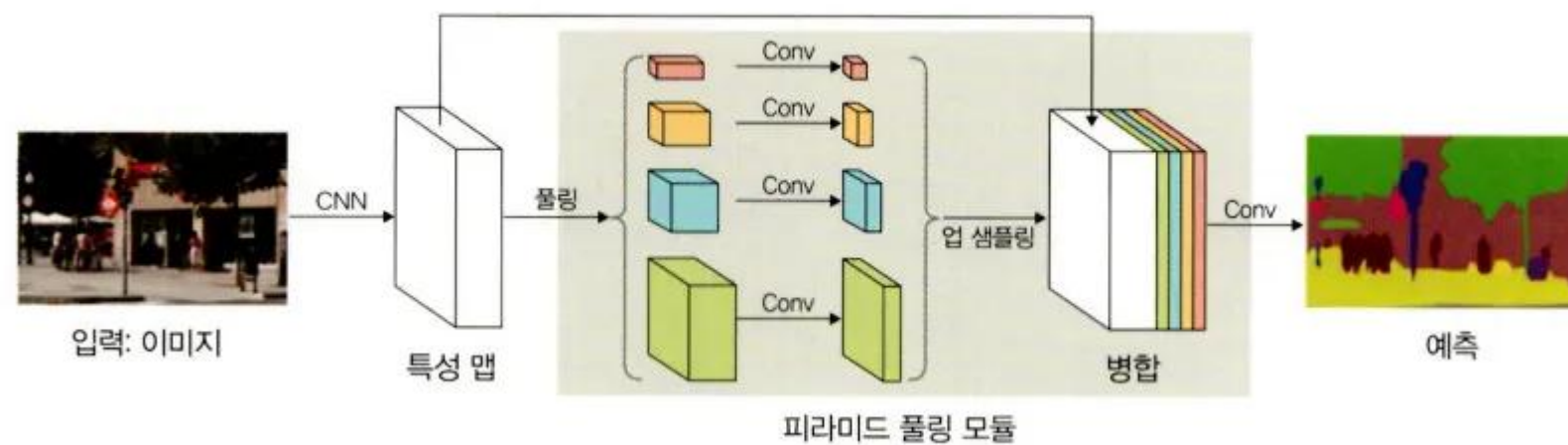
▼ 그림 6-51 U-Net



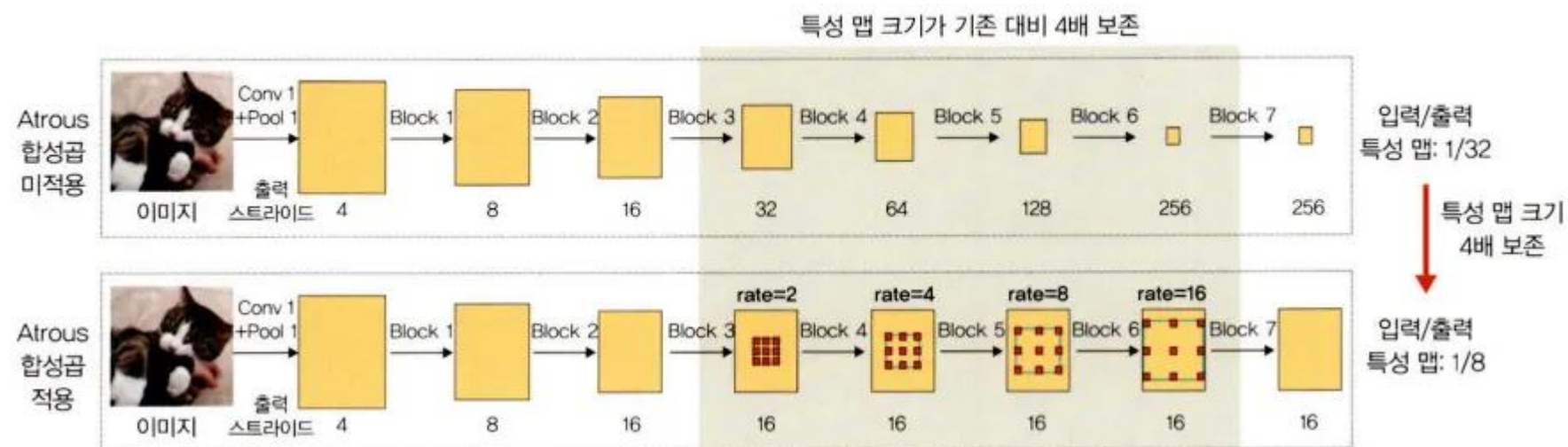
#03-2 PSPNet & DeepLabv3+

- PSPNet: 피라미드 풀링으로 넓은 수용 영역 확보
- DeepLabv3+: Atrous 합성곱으로 파라미터 수 증가 없이 수용 영역 확대
- 특징: 인코더-디코더 구조 + 고해상도 특성 맵 유지

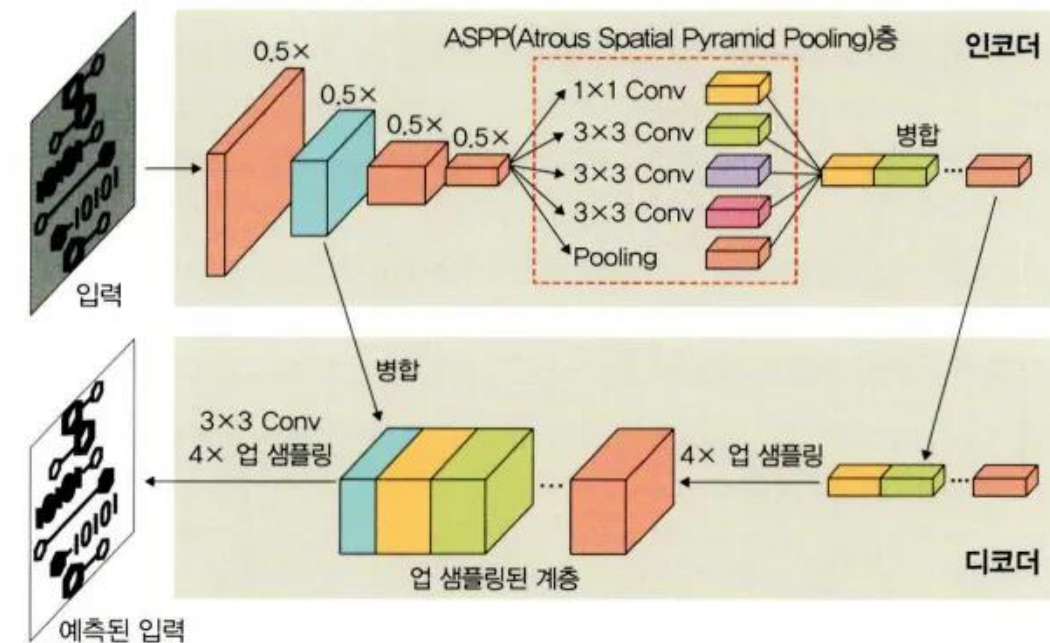
▼ 그림 6-52 PSPNet



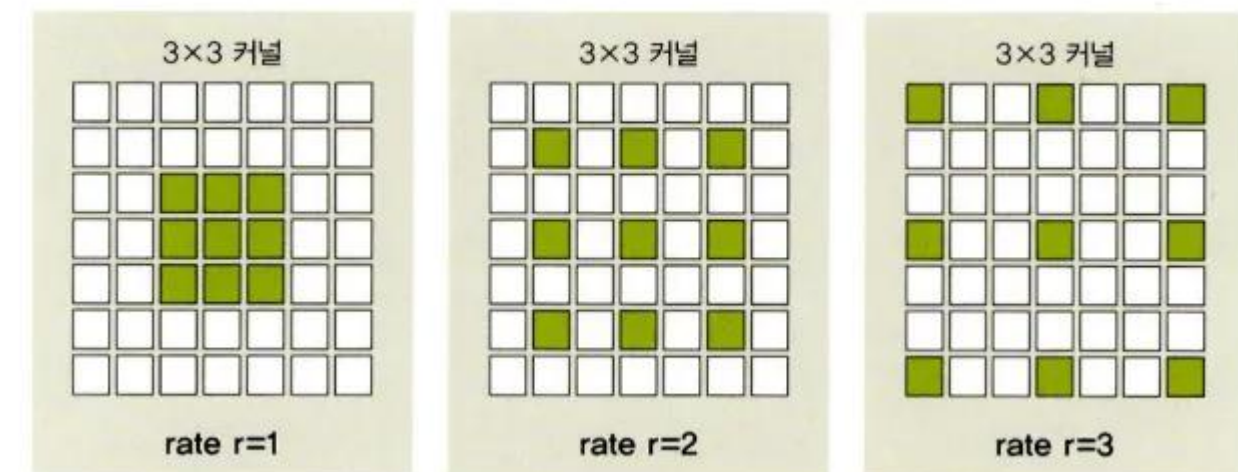
▼ 그림 6-56 Atrous 합성곱 효과



▼ 그림 6-54 DeepLab의 인코더-디코더 구조



▼ 그림 6-55 Atrous 합성곱



THANK YOU

